Exercise Sheet 1: Python foundations

○ Self-study † Class discussion

Class discussion: Questions 9, 10, 12, 13, 14.

Questions 14–16 use the accompanying `object_oriented.py`.

Question 1 - Functions ○

The volume of a sphere with radius r is $V = \frac{4}{3}\pi r^3$. Use 3.14159 for π, or import `pi` from `math`.

Save the following template as `sphere_volume.py`. Complete the docstring and replace `pass` with a return statement that calculates the volume. Keep the calculation on one line.

```
def sphere_volume(r):
    """Complete this docstring."""
    # Replace pass with a return statement.
    pass


if __name__ == "__main__":
    print("r = 0:", sphere_volume(0.0))
    print("r = 1:", sphere_volume(1.0))
```

Run `python3 sphere_volume.py` in the terminal. The main block calls your function with two test inputs and prints the results. Check that radius 0 gives 0 and radius 1 gives approximately 4.18879.

When the file is run directly, Python sets `__name__` to `"__main__"`, so the test block runs. If another file imports `sphere_volume`, the function is available but the test block is skipped.

Use a short docstring to describe the inputs and return value in this and the following questions.

Question 2 - Summary statistics ○

Write a function `summary_statistics(L)` that accepts a non-empty list of numbers and returns the minimum, maximum, and average (in that order). Can you implement this function in a single line?

Test your function on `[1, 2, 3, 4]`, `[-3, -1, 2, 6]`, and `[5]`. Work out the expected results by hand, then compare them with the returned values. Include these tests in an `if __name__ == "__main__":` block.

Question 3 - FizzBuzz ○

Write a list of the integers from 1 to 100 (inclusive) in which any number divisible by three is replaced by the word “fizz” and any divisible by five by the word “buzz”. Numbers divisible by both become “fizz buzz”. Can you create a numpy array containing this list?

Question 4 - String slicing ○

Write two new functions manipulating strings, called `first_half()` and `backward()`.

1. `first_half()` should accept a parameter and return the first half of it, excluding the middle character if there is an odd number of characters. (Hint: the built-in function `len()` returns the length of the input.)
2. The `backward()` function should accept a parameter and reverse the order of its characters using slicing, then return the reversed string. (Hint: The `step` parameter used in slicing can be negative.)

Use IPython/Jupyter to quickly test your syntax for each function.

Question 5 - Lists ◦

Write a function called `list_ops()`. Define a list with the entries `"bear"`, `"ant"`, `"cat"`, and `"dog"`, in that order. Then perform the following operations on the list:

1. Append `"eagle"`.
2. Replace the entry at index 2 with `"fox"`.
3. Remove (or pop) the entry at index 1.
4. Sort the list in reverse alphabetical order.
5. Replace `"eagle"` with `"hawk"`. (Hint: the list's `index(val)` method may be helpful.)
6. Add the string `"hunter"` to the last entry in the list.

Return the resulting list.

Work out (on paper) what the result should be, then check that your function returns the correct list. Consider printing the list at each step to see the intermediate results.

Question 6 - Comprehensions and iterators ◦

Write one-line Python expressions for (a)–(c).

a) Construct the set $S = \{x \in \mathbb{Z} \mid 0 \leq x \leq 100,\ x \bmod 3 = 1\}$.

b) Use a list comprehension to construct the pairs (i, j) with $i \in \{1, 2, 3, 4\}$ and $j \in \{5, 7, 9\}$.

c) Invert a dictionary whose values are unique and hashable, so that each value becomes a key and each key becomes its value.

d) What does each expression below return? If an expression returns an iterator, convert it to a list to see its values.

```
map(lambda x: x % 3, range(5))
list(filter(lambda x: x % 2 == 0, range(5)))
```

Question 7 - Pig Latin ◦

Write a function called `pig_latin()`. Accept a parameter `word`, translate it into Pig Latin, then return the translation. Specifically, if `word` starts with a vowel, add `"hay"` to the end; if `word` starts with a consonant, take the first character of `word`, move it to the end, and add `"ay"`. (Hint: use the `in` operator to check if the first letter is a vowel.)

Question 8 - Palindromes ◦

A palindromic number reads the same both ways. The largest palindrome made from the product of two 2-digit numbers is $9009 = 91 \times 99$. Write a function called `palindrome()` that finds and returns the largest palindromic number made from the product of two 3-digit numbers.

Question 9 - Approximating pi †

We can approximate π using either random samples or a finite sum. Implement both methods and compare their errors. Assume N is a positive integer.

(a) Monte Carlo: points in a quarter-circle

The unit circle has equation $x^2 + y^2 = 1$, radius 1 and area π. Its part in the first quadrant lies inside the square $[0, 1] \times [0, 1]$. The square has area 1 and the quarter-circle has area $\pi/4$.

Choose N points independently and uniformly in the square: for each point, draw x and y independently from $[0, 1)$. A point is inside the quarter-circle when $x^2 + y^2 \leq 1$. If M of the N points are inside, their fraction estimates the area ratio:

$$\frac{M}{N} \approx \frac{\pi}{4}, \qquad \widehat{\pi}_{\mathrm{MC}} = 4\frac{M}{N}.$$

Write `estimate_pi_mc(N)` to generate the points, count those inside the quarter-circle, and return the estimate. You may use `random.random()` or a NumPy random generator; both can draw uniform samples from $[0, 1)$.

(b) Series: add alternating terms

The Leibniz series gives

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \cdots = \sum_{k=0}^{\infty} \frac{(-1)^k}{2k+1}.$$

Using only the first N terms gives

$$\widehat{\pi}_{\text{series}} = 4 \sum_{k=0}^{N-1} \frac{(-1)^k}{2k+1}.$$

The signs alternate and the denominators are the odd integers. For example, $N = 3$ gives $4(1 - 1/3 + 1/5)$. Write `estimate_pi_series(N)` to return this finite-sum approximation. You may use a loop or `sum()` with a comprehension.

(c) Compare the approximations

For $N = 100$, 1000, and 10000, record both estimates and their absolute errors $|\widehat{\pi} - \texttt{math.pi}|$, using `math.pi` as the reference value. Repeat the Monte Carlo calculation five times for each N. Do the estimates remain the same?

Question 10 - Letter frequencies †

Write a function `letter_frequencies(text)` that returns a dictionary containing a counter for each of the 26 English letters, `a` through `z`. Include letters that do not occur, with count zero.

1. Start with all 26 counters set to zero. You may use the string `"abcdefghijklmnopqrstuvwxyz"` or `string.ascii_lowercase` to obtain the letters.
2. Treat uppercase and lowercase as the same letter: convert the input with `.lower()`. Ignore spaces, punctuation, digits and any characters outside `a`–`z`.
3. Loop through the converted string once and update the corresponding counter for each English letter.
4. In an `if __name__ == "__main__":` block, call your function on a string and print the 26-letter frequency table in alphabetical order. Then print the most frequent letter and its count.

If several letters share the largest count, choose the alphabetically first. If there are no English letters, print `"No letters"` instead of selecting a letter.

Test cases:

- `"Banana!"`: `a` occurs 3 times, `n` twice, and `b` once; all other counts are zero. The most frequent letter is `a`, with count 3.
- `"cab"`: all three letters occur once; choose `a` because it comes first alphabetically.
- `"123!"`: all counts are zero; print `"No letters"`.

Your function should return the counts; printing belongs in the main block.

Question 11 - Mutable and immutable objects ◦

Determine which types are mutable: `int`, `str`, `list`, `tuple`, and `set`.

For each type:

1. Create an object named `a`, then write `b = a`. Check `a is b`.
2. Operate on `b`: use `b += 1` for an integer, `b += "!"` for a string, `b.append(3)` for a list, `b += (1,)` for a tuple, or `b.add(3)` for a set.

3. Print `a`, `b`, and `a is b`. Did the operation change the shared object, or rebind `b` to a different object?

Assignment gives the same object a second name. Mutating it can affect both names; rebinding one name leaves the other unchanged. Use identity (`is`) as well as values to distinguish these cases.

For example:

```
a = {1: "x", 2: "b"}
b = a
b[1] = "a"
print(a, b, a is b)
```

State which types are mutable and which are immutable.

Question 12 - Modules †

A module is a Python file whose functions and other names can be imported and reused in another file. Here you will create your own module and use it in a separate program. Save both files in the same folder.

(a) Create `calculator.py` using this template. Complete `add()` and `multiply()` so they return the sum and product of their two inputs. The import makes `sqrt` available through this module as well.

```
# calculator.py
from math import sqrt


def add(a, b):
    # Return the sum.
    pass


def multiply(a, b):
    # Return the product.
    pass
```

Keep this file to imports and function definitions: it should not print results or call the functions when run or imported.

(b) Create a second file, `triangle.py`, using the template below. `import calculator` gives access to the module; for example, `calculator.add(2, 3)` calls its `add()` function. The dot specifies which module the function belongs to.

```
# triangle.py
import calculator


def hypotenuse(a, b):
    # Complete using calculator.add, calculator.multiply,
    # and calculator.sqrt. Do not use +, * or ** here.
    pass


if __name__ == "__main__":
    print(hypotenuse(3.0, 4.0))
    print(hypotenuse(5.0, 12.0))
```

For positive side lengths a and b, the hypotenuse is $c = \sqrt{a^2 + b^2}$. Use only functions accessed through `calculator` to compute and return it.

(c) Run `python3 triangle.py` and check the results.

Question 13 - Power sets †

The power set of a set A, denoted $\mathcal{P}(A)$ or 2^A, is the set of all subsets of A, including the empty set $\emptyset$ and A itself. For example, the power set of the set $A = \{a, b, c\}$ is $2^A = \{\emptyset, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$.

Write a function that accepts an iterable A and uses an `itertools` function to compute the power set of A as a list of sets (why couldn't it be a set of sets in Python?). (Hint: The power set of a set with n elements should have exactly 2^n elements.)

Question 14 - Classes †

Expand the `Backpack` class from the file object_oriented.py to match the following specifications. Complete this class before continuing with Questions 15–16.

1. Modify the constructor so that it accepts three total arguments: `name`, `color`, and `max_size` (in that order). Make `max_size` an optional parameter with default value 5. Store each input as an attribute.
2. Modify the `put()` method to check that the backpack does not go over capacity. If there are already `max_size` items or more, print "No Room!" and do not add the item to the contents list.
3. Write a new method called `dump()` that resets the contents of the backpack to an empty list. This method should not receive any arguments (except `self`).
4. Documentation is especially important in classes so that the user knows what an object's attributes represent and how to use methods appropriately. Update (or write) the docstrings for the `__init__()`, `put()`, and `dump()` methods, as well as the actual class docstring (under `class` but before `__init__()`) to reflect the changes from parts 1-3 of this problem.

To ensure that your class works properly, write a test function outside the `Backpack` class that instantiates and analyzes a `Backpack` object.

```
def test_backpack():
    testpack = Backpack("Barry", "black")       # Instantiate the object.
    if testpack.name != "Barry":                # Test an attribute.
        print("Backpack.name assigned incorrectly")
    for item in ["pencil", "pen", "paper", "computer"]:
        testpack.put(item)                      # Test a method.
    print("Contents:", testpack.contents)
    # ...
```

Question 15 - Inheritance ◦

In object_oriented.py write a `Jetpack` class that inherits from the `Backpack` class.

1. Override the constructor so that in addition to a name, color, and maximum size, it also accepts an amount of fuel. Change the default value of `max_size` to 2, and set the default value of fuel to 10. Store the fuel as an attribute.
2. Add a `fly()` method that accepts an amount of fuel to be burned and decrements the fuel attribute by that amount. If the user tries to burn more fuel than remains, print "Not enough fuel!" and do not decrement the fuel.
3. Override the `dump()` method so that both the contents and the fuel tank are emptied.
4. Write clear, detailed docstrings for the class and each of its methods.

Question 16 - Special methods ◦

Endow the `Backpack` class from the file object_oriented.py that you have been developing in the two previous questions with two additional magic methods:

1. The `__eq__()` magic method is used to determine if two objects are equal, and is invoked by the `==` operator. Implement the `__eq__()` magic method for the `Backpack` class so that two `Backpack` objects are equal if and only if they have the same name, color, and number of contents.
2. The `__str__()` magic method returns the string representation of an object. This method is invoked by `str()` and used by `print()`. Implement the `__str__()` method in the `Backpack` class so that printing a `Backpack` object yields the following output (that is, construct and return the following string).

   Use `\t` for tabs and `\n` for newlines in the returned string.

```
<<Owner:      <name>
Color:      <color>
Size:       <number of items in contents>
Max Size:   <max_size>
Contents:   [<item1>, <item2>, ...]>>
```